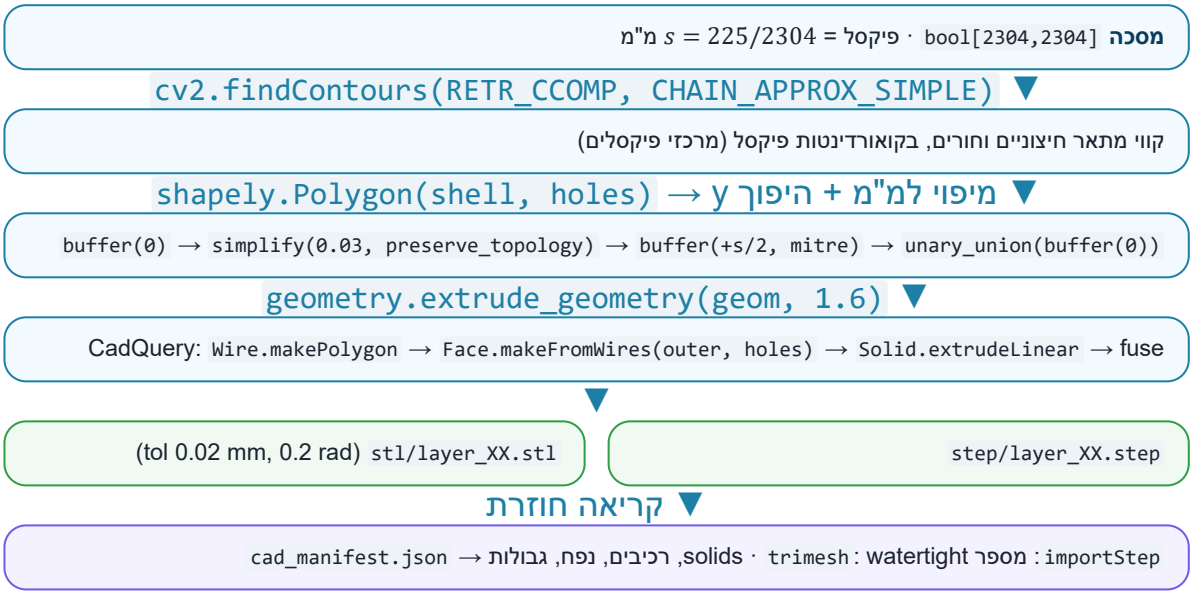
11. בניית CAD וייצוא

הסקריפט `scripts/export_reveal_stack_cad.py:53` קורא את קובצי `retained_material_mask.png` בלבד, ולא שום מצב אחר של המתכנן. לכן הגאומטריה הפיזית נגזרת בדיוק מהמסכה שעברה אימות.

11.1 תרשים 5: מסכה → מוצק



11.2 mask_to_geometry: ממסכה לפוליגונים

`scripts/export_reveal_stack_cad.py:29`

```python
scripts/export_reveal_stack_cad.py:29-50
def mask_to_geometry(mask: np.ndarray, mm_per_pixel: float, simplify_mm: float):
    """Polygons whose edges follow pixel edges (not pixel centres)."""

    contours, hierarchy = cv2.findContours(
        mask.astype(np.uint8), cv2.RETR_CCOMP, cv2.CHAIN_APPROX_SIMPLE
    )
    height = mask.shape[0]
    to_mm = lambda c: [((x + 0.5) * mm_per_pixel, (height - y - 0.5) * mm_per_pixel) for x, y in c[:, 0]]
    shells = []
    for index, contour in enumerate(contours):
        if hierarchy[0][index][3] != -1 or len(contour) < 3:
            continue
        holes = []
        child = hierarchy[0][index][2]
        while child != -1:
            if len(contours[child]) >= 3:
                holes.append(to_mm(contours[child]))
            child = hierarchy[0][child][0]
        shells.append(Polygon(to_mm(contour), holes))
    half = 0.5 * mm_per_pixel
    geometry = unary_union([s.buffer(0) for s in shells]).buffer(half, join_style=2, mitre_limit=2.0)
    return geometry.simplify(simplify_mm, preserve_topology=True).buffer(0)
```

שלב א: מעקב קווי מתאר

`RETR_CCOMP` מחזיר היררכיה בשתי רמות: קווי מתאר חיצוניים (ההורה הוא −1) וחורים, שהם הילדים שלהם, ונעבור עליהם בעזרת `hierarchy[...][2]` (ילד ראשון) ו־`[0]` (אח הבא). אי שבתוך חור מופיע שוב כקו מתאר חיצוני ברמה העליונה. `CHAIN_APPROX_SIMPLE` משאיר רק את קצוות הקטעים הישרים (אופקיים, אנכיים ואלכסוניים). OpenCV עוקב אחרי **מרכזי** פיקסלי הגבול של החומר, גם בחורים.

שלב ב: מעבר למ"מ והיפוך ציר

IMPLEMENTED MATHEMATICS שורה 36:

$$X = (x + 0.5)s, \qquad Y = (H - y - 0.5)s, \qquad H = 2304,\ s = 0.09765625\ \text{mm}$$

פיקסל (x, y) תופס ב־CAD את התיבה $[xs, (x+1)s] \times [(H-y-1)s, (H-y)s]$, ו־$(X, Y)$ הוא מרכזה. y של התמונה גדל כלפי מטה, ו־Y של ה־CAD כלפי מעלה, ולכן יש היפוך. **דוגמה:** הפיקסל השמאלי־עליון $(0, 0)$ ממופה ל־$(0.0488,\ 224.95)$ מ"מ.

שלב ג: הרחבה בחצי פיקסל

DERIVED הפוליגון עובר דרך מרכזי פיקסלי הגבול, ולכן הוא קטן בחצי פיקסל מהמסכה האמיתית בכל צד. `buffer(s/2, join_style=2, mitre_limit=2.0)` מזיז כל צלע החוצה ב־$s/2$ עם פינות חדות (mitre). בצלעות אופקיות ואנכיות, הגבול עובר בדיוק לקצה הפיקסל. בחור, הקו עובר דרך מרכזי החומר שסביבו, וההרחבה מזיזה אותו **לתוך** החור, שוב בדיוק לקצה. בצלעות אלכסוניות של `CHAIN_APPROX_SIMPLE` התוצאה היא קו ישר ולא מדרגות, ולכן יש סטייה תת־פיקסלית.

שלב ד: פישוט Douglas–Peucker

THEORETICAL BACKGROUND `simplify(ε=0.03, preserve_topology=True)` מסיר קודקודים כל עוד המרחק של כל קודקוד שהוסר מהמיתר החדש קטן מ־ε, כלומר 0.03 מ"מ ≈ 0.31 פיקסל. `preserve_topology` מבטיח שטבעות לא יחצו זו את זו ושחורים לא ייעלמו. `buffer(0)` בסוף מתקן פוליגונים לא תקינים.

IMPLEMENTED MATHEMATICS בדיקת השטח (שורה 93):

$$\Delta_A = 100 \cdot \frac{A_{\text{poly}} - n_{\text{px}} s^2}{n_{\text{px}} s^2}\ \%$$

ב־Option B: בין 0.0% (הלוח האחורי) ל־0.085% (הלוח הקדמי, עם 10,595 קודקודים).

LIMITATION בשורה 39 קווי מתאר עם פחות מ־3 נקודות מושמטים. רכיב של פיקסל בודד או שני פיקסלים היה נעלם. מסכה שעברה את בדיקה 1 היא רכיב אחד גדול, ולכן זה לא משפיע כאן, אבל זה עלול להשפיע על חורים זעירים.

extrude_geometry: CadQuery / OpenCascade 11.3

`src/layerforge_l1/geometry.py:97` (קוד קיים, ללא שינוי)

```python
# src/layerforge_l1/geometry.py:84-109
def _wire(coords: Iterable[tuple[float, float]]) -> cq.Wire:
    points = [cq.Vector(float(x), float(y), 0.0) for x, y in coords]
    if points[0].toTuple() != points[-1].toTuple():
        points.append(points[0])
    return cq.Wire.makePolygon(points)


def _face(poly: Polygon) -> cq.Face:
    exterior = _wire(poly.exterior.coords)
    interiors = [_wire(ring.coords) for ring in poly.interiors]
    return cq.Face.makeFromWires(exterior, interiors)


def extrude_geometry(geometry: BaseGeometry, height_mm: float, z0_mm: float = 0.0) -> cq.Shape:
    solids: list[cq.Shape] = []
    for poly in polygons(geometry):
        solid = cq.Solid.extrudeLinear(_face(poly), cq.Vector(0.0, 0.0, height_mm))
        if z0_mm:
            solid = solid.translate(cq.Vector(0.0, 0.0, z0_mm))
        solids.append(solid)
    if not solids:
        raise ValueError("cannot extrude geometry without polygonal area")
    result = solids[0]
    for solid in solids[1:]:
        result = result.fuse(solid)
    return result
```

IMPLEMENTED MATHEMATICS

$$\text{Solid} = \text{Face}(\partial_{\text{out}}, \{\partial_{\text{hole}}\}) \times [z_0,\ z_0 + t], \qquad t = 1.6\ \text{mm},\ z_0 = 0, \qquad V = A \cdot t$$

החורים הם wires פנימיים של ה־face, ולכן אחרי ה־extrusion הם **חורים אמיתיים** שעוברים דרך כל העובי, כמו שנדרש ב־AGENTS.md. אם יש כמה פוליגונים, הם מאוחדים ב־`fuse`. ב־Option B יש פוליגון אחד בכל לוח, ולכן אין צורך ב־fuse. העובי 1.6 מ"מ הוא DESIGN_CHOICE, ברירת המחדל של `--thickness-mm`.

11.4 ייצוא ובדיקת קריאה חוזרת

```python
# scripts/export_reveal_stack_cad.py:72-80
        solid = extrude_geometry(geometry, args.thickness_mm)
        stem = layer_dir.name
        step_path, stl_path = out / "step" / f"{stem}.step", out / "stl" / f"{stem}.stl"
        cq.exporters.export(solid, str(step_path), exportType="STEP")
        cq.exporters.export(solid, str(stl_path), exportType="STL", tolerance=0.02, angularTolerance=0.2)
        t_export = time.perf_counter()
        step_solids = len(cq.importers.importStep(str(step_path)).val().Solids())
        mesh = trimesh.load_mesh(stl_path, file_type="stl", process=True)
        components = len(mesh.split(only_watertight=False))
```

בדיקה	חישוב	מה זה מוכיח
step_solids	מספר ה־Solids ב־`importStep`	שהקובץ נפתח ושיש בו מוצק אחד ולא רסיסים
stl_watertight	`mesh.is_watertight`	שכל צלע משותפת לשני משולשים בדיוק, כלומר רשת סגורה
stl_components	`len(mesh.split())`	שאין איים ברשת
volume_vs_area_x_thickness_pct	$100(V_{\text{mesh}} - At)/(At)$	שהמשולשים תואמים לפוליגון
stl_bounds_mm	`mesh.bounds`	שהלוח בגודל 225 × 225 × 1.6 ובמקום הנכון

DERIVED כל הצלעות ישרות (אין קשתות), ולכן טולרנס ה־tessellation של STL (0.02 מ"מ) לא משנה את הגאומטריה. כל פאה מתחלקת למשולשים בלי קירוב. זה מתאים לסטייה של 0.0% בנפח.

11.5 קואורדינטות הרכבה וקואורדינטות הדפסה

```python
scripts/export_reveal_stack_cad.py:101-116
    order = sorted(rows, key=lambda r: r["layer"])  # layer_00 = back
    manifest = {
        "outer_sheet_mm": args.outer_mm,
        "thickness_mm": args.thickness_mm,
        "assembly_back_to_front": [
            {
                "layer": r["layer"],
                "color": r["color"],
                "stl": f"stl/{r['layer']}.stl",
                "step": f"step/{r['layer']}.step",
                "assembled_z0_mm": i * args.thickness_mm,
            }
            for i, r in enumerate(order)
        ],
        "print_placement": "every STL at z = 0, broad face on the bed",
        "layers": rows,
```

- **הדפסה:** כל קובץ STL/STEP נמצא ב־$z \in [0, 1.6]$, עם הפאה הרחבה על המשטח, ו־XY זהים בכל הלוחות (רישום משותף).
- **הרכבה:** רק מספר ב־manifest: $z_0^{(i)} = 1.6i$, כאשר i=0 הוא הלוח האחורי (`layer_00`) ו־i=9 הקדמי, ב־14.4 מ"מ.
- GAP לא נוצר קובץ הרכבה תלת־ממדי ולא תצוגה מפוצצת (exploded) **מתוך המוצקים**. ההרכבה שהוצגה (`stack_from_cad.png`) נבנתה מרסטור מחדש של הפוליגונים, כלומר מהגאומטריה שנכנסה ל־CAD, ולא מקריאת קובצי STEP/STL עצמם.

11.6 אימות חוזר אחרי CAD AD HOC

הקוד המלא שהורץ (אינו במאגר):

```
N=2304; mm=225/N; S=4
def raster_ss(geom):                          # 4x supersampled, 1/16 sub-pixel precision
    img=np.zeros((N*S,N*S),np.uint8)
    f=lambda cs: np.round(np.array([(x/mm*S,(N-y/mm)*S) for x,y in cs])*16-8).astype(np.int32)
    for poly in polygons(geom):
        cv2.fillPoly(img,[f(poly.exterior.coords)],1,shift=4)
        for r in poly.interiors: cv2.fillPoly(img,[f(r.coords)],0,shift=4)
    return img.reshape(N,S,N,S).sum((1,3))>=(S*S)/2
for d in layers (front first):
    m = retained mask; v = visible mask
    r = raster_ss(mask_to_geometry(m, mm, 0.03))
    edge = audit_edge_connectivity(r); width = audit_connection_width(r, (v&r)[p:-p,p:-p], grid,
DEFAULT_OPTIMIZATION_CONFIG)
    px_diff = (r != m).sum(); cad_stack[r & ~shown] = layer colour; shown |= r
stack_mismatch = (cad_stack != mask_derived_composite) inside the artwork
```

IMPLEMENTED MATHEMATICS

$$u = \frac{X}{s}S, \quad v = \left(H - \frac{Y}{s}\right)S, \qquad r(p) = \left[\sum_{i,j<4} \mathrm{in}(p,i,j) \ \geq 8\right]$$

הפוליגון נצבע ברשת פי 4, עם דיוק של 1/16 פיקסל (`shift=4`; ההיסט 8− הוא חצי פיקסל). פיקסל נחשב "מלא" אם לפחות חצי מ־16 תתי־הפיקסלים שלו בפנים. **תוצאות** (OBSERVED, `cad_revalidation.json`): לכל 10 הלוחות 0 פיקסלים שונים מהמסכה, neighbor PASS, width PASS-4, מסגרת מלאה. `stack_mismatch_px = 0`.

ניסיון ראשון עם `fillPoly` רגיל (בלי supersampling) הראה הפרשים, אבל הם נבעו מכלל המילוי של ה־rasterizer ולא מהגאומטריה. ההרצה המחודשת ב־supersampling הראתה 0 הפרשים, ורק היא נשמרה בקובץ.